

FAISS_Analysis 交付报告

版本: v1.0

日期: 2025-10-14

适用文件: FAISS_Analysis.py

1. 项目概述

本脚本用于在两个常见数据集（FMNIST 与 Inshop）上，进行 FAISS 检索配置的网格搜索与可视化分析，覆盖两类索引：

- **IVF（倒排文件）**：支持 PCA → IVF, Flat 链式结构，暴露 `pca_dim` / `nlist` / `nprobe` 三个主参；
- **HNSW**：暴露 `M` / `efConstruction` / `efSearch` 三个主参。

输出包括：

- 构建时间 `build_time_s`
- 查询时间 `query_time_ms`（按 k 的 batch 平均）
- 索引体积 `index_size_mb`
- 向量级召回 `vector_recall_at_k`（与 Flat-Gallery Top-k 的重合度）
- 标签级精度/召回（micro & macro）`label_precision_at_k_micro` / `label_recall_at_k_micro`、`precision_at_k_macro` / `recall_at_k_macro`

2. 数据与路径

脚本通过 `PROJECT_DIR` 约定数据目录结构，并提供两套路径宏：

- **FMNIST**
 - `FMNIST_VECTOR_GALLERY` / `FMNIST_LABEL_GALLERY`
 - `FMNIST_VECTOR_QUERY` / `FMNIST_LABEL_QUERY`
- **Inshop**
 - `INSHOP_VECTOR_GALLERY` / `INSHOP_LABEL_GALLERY_NPY`
 - `INSHOP_VECTOR_QUERY` / `INSHOP_LABEL_QUERY_NPY`

说明：IVF 流程使用 **gallery-only** 建索引，**query-only** 做评估；HNSW 流程默认使用单集（`FAISS_NeighborSearch.VECTORS_PATH`）并按标签分层抽样 query。

3. 主要函数与职责

3.1 建索引

- `create_ivf_index(vectors_path, index_path, nlist, nprobe, pca_dim, metric)`
 - 通过 `faiss.index_factory` 组合 `"PCA{pca_dim}, IVF{nlist}, Flat"` 或 `"IVF{nlist}, Flat"`
 - 统一设置 `nprobe`；返回索引实例并打印元信息

- `create_index_hnsw(vectors_path, index_path, M, efConstruction, efSearch, metric)`
 - 通过 `faiss.index_factory` 组合 `"HNSW{M},Flat"`
 - 设置 `efConstruction / efSearch`；返回索引实例并打印元信息

3.2 评估与度量

- `grid_search_ivf(nlist_param, nprobe_param, pca_dim_param, ..., k, n_per_label, save_csv, ...paths)`
 - 流程：
 1. 加载 gallery/query 向量与标签
 2. 在 gallery 上构建 **Flat 基线**，得到 `Ig` (Top-k 真值参考)
 3. 网格遍历 (`pca_dim, nlist, nprobe`)：
 - 构建 IVF 索引并计时、落盘测体积
 - 查询 Query 批次，计算：
 - `vector_recall_at_k`：与 `Ig` 的交集占比
 - `label_precision_at_k_micro / label_recall_at_k_micro`：逐 query 的精度/召回后做 micro 平均
 - `precision_at_k_macro / recall_at_k_macro`：逐 label 的宏平均
 4. 汇总为 `df` (整体) 和 `df_label` (按标签)，可选保存 CSV
 - 输出：(`df, df_label`)
- `grid_search_hnsw(M_param, efC_param, efS_param, ..., k, n_per_label, save_csv)`
 - 对 (`M, efConstruction, efSearch`) 进行网格搜索
 - 通过 `Flat.reconstruct(qid)` 获取 query 向量，计算 `recall_at_k` (标签命中率)，并统计 `build_time_s / query_time_ms / index_size_mb`
 - 输出：`df`

3.3 可视化

- IVF 系列：
 - `plot_ivf_build_and_size_panels(df, dataset_title)`：双面板 (Build/Size)，维度 `pca_dim × nlist`
 - `plot_ivf_querytime_by_pca(df, dataset_title)`：`nprobe × nlist` (按 PCA 分面)
 - `plot_ivf_vector_recall_by_pca(df, dataset_title, k_val)`
 - `plot_ivf_label_precision_by_pca(df, dataset_title, k_val)`
 - `plot_ivf_label_recall_by_pca(df, dataset_title, k_val)`
- HNSW 系列：
 - `plot_hnsw_build_and_size(df)`：双面板 (Build/Size)，维度 `efConstruction × M`
 - `plot_hnsw_querytime(df)`：`efSearch × M`
 - `plot_hnsw_recall(df, k_val)`：`efSearch × M`

可视化中提供单元格注释 `_annotate_cells`，便于快速阅读均值热力图。

4. 运行与复现实验

4.1 环境依赖

```
pip install numpy pandas matplotlib faiss-cpu
# 或 faiss-gpu (若使用 GPU 版本)
```

4.2 运行 IVF (FMNIST)

```
if __name__ == "__main__":
    nlist = [256, 512, 1024]
    nprobe = [1, 4, 8, 16, 32]
    pca_dim = [None, 64, 128, 256, 512, 1024]
    K = 20

    df, df_label = grid_search_ivf(
        nlist_param=nlist,
        nprobe_param=nprobe,
        pca_dim_param=pca_dim,
        indexBuilder=create_ivf_index,
        k=K,
        n_per_label=None, # 默认使用全部 query
        save_csv="ivf_FMNIST_query_metrics.csv",
    )

    plot_ivf_build_and_size_panels(df, "FMNIST")
    plot_ivf_querytime_by_pca(df, "FMNIST")
    plot_ivf_vector_recall_by_pca(df, "FMNIST", k_val=K)
    plot_ivf_label_precision_by_pca(df, "FMNIST", k_val=K)
    plot_ivf_label_recall_by_pca(df, "FMNIST", k_val=K)
    plt.show()
```

4.4 运行 HNSW

取消 HNSW 区块注释并运行，得到 `df_hnsw` 与三张热力图。

5. 指标定义与注意事项

- `vector_recall_at_k`：对每个 query，ANN 与基线（gallery-Flat）Top-k 的交集比例，再取平均；衡量近似搜索的向量级重合率。
- `label_precision_at_k_micro`：每 query 的 Top-k 中正确标签占比，直接平均（micro）；固定 `k` 时可与 `label_recall` 等价讨论（见下）。
- `label_recall_at_k_micro`：每 query 的 Top-k 命中数 / （该标签在 gallery 的总数），再平均。
- **Macro 版本**：先按 label 聚合，再做均值，缓解长尾类被淹没。

等价性提醒：若每个 query 的 GroundTruth 数量 = 评估的 `k`（例如 GT=20 且取 top-20），则 `precision@20 = recall@20 = 命中数 / 20`。二者在数值上相等；可只保留一个指标以简化评估。

6. 性能与规模

- 构建时间：
 - IVF: 受 `nlist` 与数据规模主导；`pca_dim` 在实际数据中对 `build_time_s` 影响常呈弱相关（常数/IO 占主导）。
 - HNSW: 随 `M / efConstruction` 增长而上升。
- 查询时间：
 - IVF: 随 `nprobe` 增大近似线性上升；`nlist` 影响次之。
 - HNSW: 随 `efSearch` 增大近似单调上升。
- 索引体积：
 - IVF: 主要由 `nlist` 与是否启用 `PCA` 决定；在某些配置下随 `pca_dim` 呈次线性增长（与实现/量化相关）。
 - HNSW: 随 `M` 增长近似线性。

实测趋势受具体数据分布影响；建议结合热图与表格对比。

7. 常见问题（FAQ）

1. 为什么 IVF 的向量召回不是 1？
因为 ANN 近似搜索未遍历全量倒排桶，`nprobe` 较小时易漏检。
2. `pca_dim=None` 与 `pca_dim=0` 有何区别？
实现中 `None/0` 都表示不启用 `PCA`，即 `IVF{nlist}, Flat`。
3. 能否只采样一部分 query？
`grid_search_ivf` 支持 `n_per_label` 分层采样；HNSW 的 `grid_search_hnsw` 通过 `_sample_query_ids` 控制样本量。

8. 结果产物

- CSV（可选保存）
 - `ivf_*.csv`：整体结果
 - `ivf_*_per_label.csv`：每标签的 macro 指标
- 图像
 - IVF: 5 张（Build/Size 双面板 + QueryTime + VectorRecall + Label Precision + Label Recall）

9 风险与边界

- 若数据集标签极度不均衡，micro 指标可能过分偏向大类；
- `Flat` 基线用于生成向量级 Top-k 参考，如基线与评估集合不一致会偏置 `vector_recall_at_k`；
- 文件写入体积因文件系统与序列化方式可能有少量抖动。